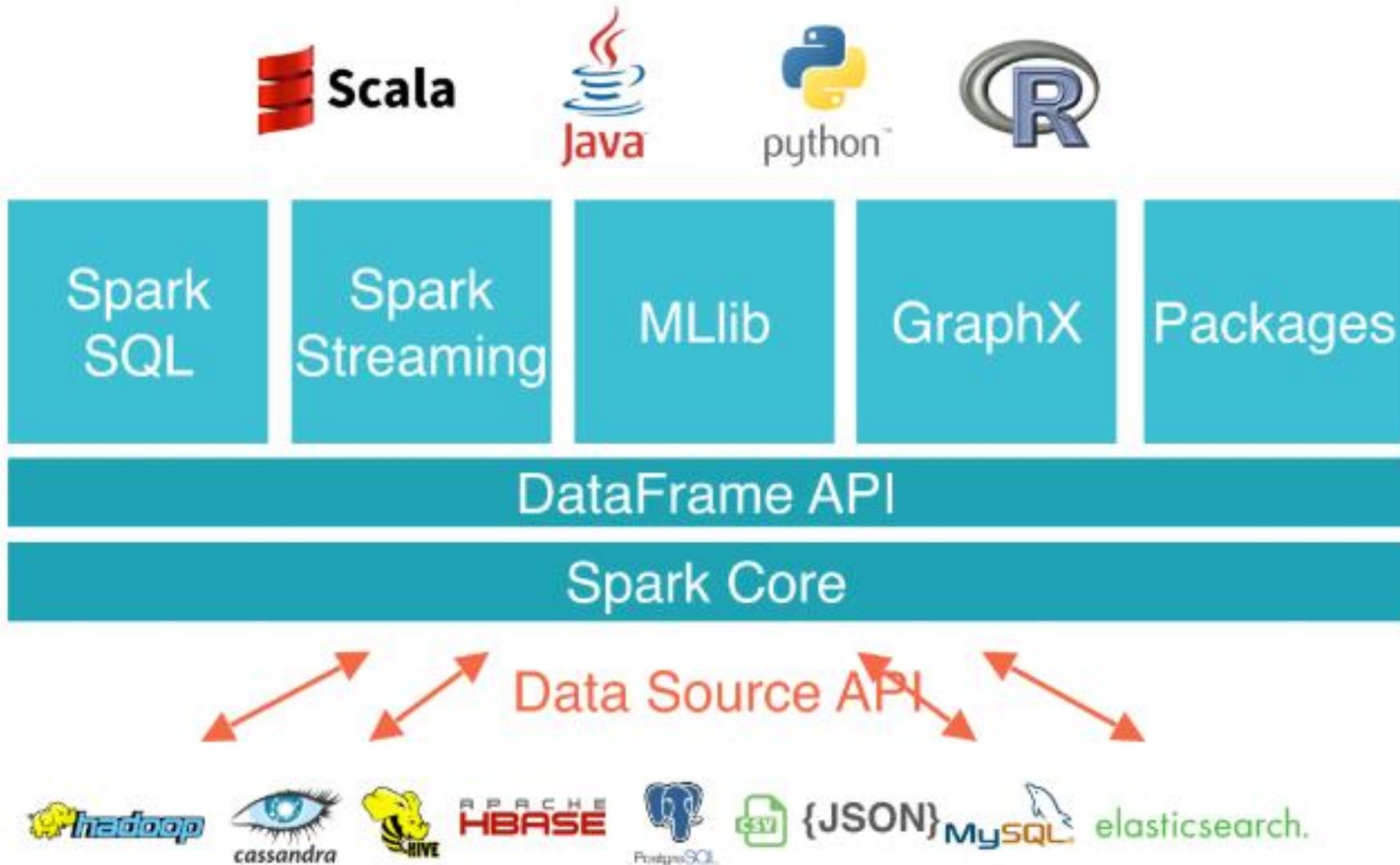


Spark and Hadoop

- Spark supports distributed datasets from any files stored in the Hadoop Distributed File System (HDFS).
- Spark can access Hive tables (it is the default!)
- Spark can access Hbase tables
- Spark can connect to MySQL db and other dbs through ODBC/JDBC
- **Spark can access the local files without HDFS**

Spark Key Components



Spark SQL, Datasets, and DataFrames

- Spark SQL is a Spark module for structured data processing.
 - Spark SQL can also be used to read data from an existing Hive installation
 - **When running SQL from within another programming language the results will be returned as a [Dataset/DataFrame](#)**
 - Execute Spark SQL command in command line or over ODBC/JDBC
 - Speed of spark SQL is comparable to that of Impala
- A DataFrame is a Dataset organized into named columns
 - It is conceptually equivalent to a table in Relational DB or data frame in Python
 - Dataframe API is available in Python
- A Dataset is a distributed collection of data.
 - RDD: Resilient Distributed Dataset
 - The dataset API is available in Scala and Java only.
 - Python does not have the support for Dataset API

Three APIs @ K-State VM1

- spark-sql
 - Create a table
 - Load data into a table
 - Use SQL commands
- Pyspark
 - A python interface that can run similar RDD transformation/action and SQL functions
- spark-shell
 - Create a new RDD e.g. read file from local filesystem and create an RDD
 - Filter Operation.
 - Transformation and Action
 - Read the first item (or 5 items) from the RDD

Reference

- Chaps 4, 5, 6, 9 (Data Sources) of Chamber and Zaharia (2018)
- Chapter 4 of Bengfort and Kim (2016), Data Analytics with Hadoop – An Introduction for Data Scientists, O’Rilly.
- <https://spark.apache.org/docs/latest/sql-programming-guide.html>
- Spark Tutorial for Beginners (Comparison from Hadoop to Spark)
<https://www.youtube.com/watch?v=QaoJNXW6SQo>
- [RDD programming Guide:](#)
 - <https://spark.apache.org/docs/latest/programming-guide.html>

spark-sql demo

- Prereq
 - At vm prompt type spark-sql
 - the following prompt means it is ready: spark-sql>
 - Each SQL line can form a separate line after >
 - Enter ; after > or at the end of a statement to execute
 - The SQL statements are NOT case sensitive
 - Highlight the SQL statements in an editor, CTRL + c to copy; at spark-sql> right-click to paste
 - CTRL + c to exit the shell
- Flight data in 2015 is used for demonstration
 - A csv file with three columns available in Canvas (2015-summary.csv)
 - Upload the data file: 2015-summary.csv under VM1 /home/changs/.

Spark-sql commands (in spark-sql)

- #Create the table and load the data

```
CREATE TABLE flights (  
  DEST_COUNTRY_NAME STRING,  
  ORIGIN_COUNTRY_NAME STRING,  
  fnum LONG)  
USING csv OPTIONS (header true,  
path 'file:///home/changs/2015-summary.csv');
```

- # provide table information; note that the database is default
DESCRIBE TABLE flights;

- Show all data
SELECT * FROM flights;

Pyspark demo (in pyspark) – 1/3

- # Read a data into dataframe: flightData2015

```
flightData2015 = spark\
```

```
.read\
```

```
.option("inferSchema", "true")\
```

```
.option("header", "true")\
```

```
.csv("file:///home/changs/2015-summary.csv")
```

- #create a temporary view flight_data_2015

```
flightData2015.createOrReplaceTempView("flight_data_2015")
```

Pyspark demo – 2/3

- # using dataframe: flightData2015
dataFrameWay = flightData2015\
 .groupBy("DEST_COUNTRY_NAME")\
 .count()

Pyspark demo – 2/3

- # using temp view as table: flight_Data_2015

```
sqlWay = spark.sql("""  
SELECT DEST_COUNTRY_NAME, count(1)  
FROM flight_data_2015  
GROUP BY DEST_COUNTRY_NAME  
""")
```

Comparison of SQL way and Dataframe way

- Both ways generate the SAME spark execution plan
 - Try:

```
sqlWay.explain()
dataFrameWay.explain()
```
- SQL way uses the temp view as table in the FROM clause (e.g. the following statement would give an error message)

```
sqlWay=spark.sql("""
SELECT DEST_COUNTRY_NAME, count(1)
FROM flight_Data_2015
GROUP BY DEST_COUNTRY_NAME
""")
```
- Dataframe Way directly manipulates the dataframe

spark-shell commands (scala> Ctrl+C to quit)

check if a spark session is running
spark

#define a variable in scala

```
var myRange=spark.range(1000).toDF("number")
```

#concept of partitions: e.g. 1000 worker nodes; if a dataset is so large, it will be partitioned into multiple nodes

#transformation & lazy evaluation: to best an optimized plan

```
var evennum=myRange.where("number % 2 =0")
```

#action: to trigger a computation

```
evennum.take(5)
```

```
evennum.count()
```

Reference

- Chaps 2 of Chamber and Zaharia (2018) An-End-to-End Example
- Please note that Chamber use the name count in the third column of the flight 2015 data. The SQL function count is also a key word. In our example, we use fnum to represent the number of flights in the third columns.